

The Price of Conservatism: Empirical Evidence from Cloud Right-Sizing and Reservation Heuristics

Ibrahim Mohamed, Mariam Emad, Hazem Ibrahim, Ahmed Sameh, Mahmoud Ahmed, John Ehab

[3pt] *Academic supervision and project support: Dr. Hafez Seliem, Mohamed Essam, and Saif AlDeen Sameh.*

[3pt] All authors: Department of Computer Systems

Faculty of Computer Science

Ain Shams University

Cairo, Egypt

Abstract

Cloud right-sizing and capacity reservation answer different questions: one maps observed utilization to candidate capacity, whereas the other chooses an advance commitment under a price ratio. We use Smart Cloud Optimizer, an implemented AWS cost-optimization pipeline, as an evaluation vehicle for an empirical comparison between one implemented $q_{0.95} \times 1.3$ rule, restored public traces, and simplified reservation-policy baselines. The contribution is empirical and systems-oriented; it is not new reservation theory or a model of commercial recommenders. Study I audits price basis, catalog feasibility, and service decomposition on the repository’s synthetic account. Under a shared EC2 snapshot on-demand basis, EC2 changes from \$843.73/mo to \$970.61/mo (+15.0%), while demo-priced RDS decreases from \$397.85/mo to \$35.04/mo (91.2% decrease), yielding a service-mixed aggregate change of -19.0%. Study II restores Bitbrains and Materna GWA traces with checksums, baseline grids, and bootstrap intervals; the rule yields 41.8% lower aggregate mapped cost (95% CI 35.9–47.5%) than lift-and-shift under the evaluated catalog and mapping. Study III counterfactually reuses $P95 \times 1.3$ for reservation sizing, rather than treating it as a commercial recommender model. At $\gamma = 0.63$, it realizes 101.8% and 153.5% of on-demand-only cost on a synthetic daily account-cost proxy (USD/day) and a restored hourly aggregate CPU-demand series (MHz), respectively; the newsvendor fractile realizes 73.6% and 86.2%. These deterministic simulations evaluate relative temporal policy behavior under a simplified mapped-cost objective, not provider billing or physical-capacity allocation. Direct commercial-tool, operational, and SLO effects remain unevaluated.

Keywords: cloud cost optimization; right-sizing; capacity reservation; reserved instances; newsvendor; FinOps; utilization percentiles; time-series forecasting.

1 Introduction

Public-cloud customers pay for a mix of elastic and committed capacity. On-demand capacity is flexible but expensive per unit. Reserved or committed capacity is cheaper per unit, but unused reserved capacity still costs money. This creates a recurring decision problem for workloads with intermittent surges: reserve enough to exploit discounts, but not so much that the commitment becomes idle during ordinary periods.

The evaluated pipeline estimates high utilization percentiles and adds headroom when selecting candidate capacity. This construction maps observed usage to a capacity above the selected percentile. Published large-scale systems address distinct resource-management tasks: Autopilot

reports workload autoscaling at Google [18], Resource Central reports workload understanding and prediction for resource management [10], and Doppler reports automated SKU recommendation for SQL migration [7]. However, a capacity-sizing rule and a reservation commitment are different economic objects. The former selects mapped capacity; the latter is a financial hedge against on-demand prices. Added headroom alone is not evidence of effects on latency, errors, throughput, availability, or SLO violations, none of which are measured here.

Capacity-reservation theory makes the economic distinction explicit. Chen, Lei, and Moinzadeh [9] model cloud capacity under intermittent random demand surges and derive newsvendor-like capacity levels for reserved capacity. Their result implies that the cost-optimal reservation service level depends on the reserved/on-demand price ratio, not on an engineering convention such as the 95th percentile. At the recorded catalog-average ratio, $\gamma \approx 0.63$, the critical fractile $1 - \gamma$ is much lower than 0.95. Study III evaluates this price-ratio logic with arm-specific empirical quantities X_t and Q ; it does not reinterpret the synthetic cost series as physical demand or commitment.

This article uses Smart Cloud Optimizer as an empirical artifact for studying that distinction. The repository implements telemetry ingestion, forecasting, PuLP/CBC mixed-integer right-sizing, reserved-pricing checks, and transparent waste rules. Dashboard, API, storage, and LLM-advisor components are system context only. The evaluated object is narrower: how an implemented P95-based right-sizing pipeline behaves under corrected price bases, public traces, and price-aware reservation simulations.

The claim is not that this article introduces reservation theory or that the implemented optimizer beats AWS, Azure, or a third-party recommender. No direct evidence for such comparisons exists in the repository. The narrower claim is that the implemented pipeline and restored traces support an empirical comparison of one percentile-based sizing rule with simplified, theory-derived and recency-window reservation baselines.

This article makes three contributions.

1. It operationalizes and audits one $q_{0.95} \times 1.3$ right-sizing pipeline, separating EC2 price comparability and candidate-catalog feasibility from the demo-priced RDS contribution to the synthetic aggregate.
2. It restores GWA Study II evidence with archive and manifest checksums, bootstrap confidence intervals, and a lift-and-shift/mean/median/P90/P95/P99/ max/P95 $\times 1.3$ baseline grid.
3. It compares the deliberately conservative HEUR counterfactual with newsvendor, SAA, dual-level, perfect-information, and 7/30/60-day recency-window policies on held-out proxy series (USD/day account cost in the synthetic arm and MHz aggregate CPU demand in the restored real-trace arm), while reporting the third-party-system, contract-model, and SLO boundaries of that simulation.

The evidence boundaries remain explicit: the Azure raw `vmtable.csv.gz` file was not restored/rerun, no SLO or performance-risk measurements are available, no direct commercial-tool baseline is present, and no full dynamic (s, S) implementation is claimed.

2 Related Work

2.1 Capacity Reservation and Newsvendor Fractiles

The reservation side of this article builds on the classical newsvendor model [3], data-driven newsvendor variants [5], and stochastic cloud provisioning models that choose capacity under uncertain

demand and prices [6, 8]. Online instance-acquisition formulations similarly compare reserved and on-demand decisions over time [21, 22]. Chen et al. [9] are the closest theoretical reference: their model separates base and surge capacity and derives newsvendor-like reservation levels and dynamic policy structure. This article reuses their price-aware logic as motivation for paper-side static newsvendor and simplified base/surge comparisons; it neither extends that theory nor implements their full dynamic policy.

2.2 Practical Right-Sizing and Cloud Cost Reduction

The named systems establish different mechanisms in their respective settings: workload autoscaling in Autopilot [18], workload understanding and prediction in Resource Central [10], and automated cloud-SQL SKU recommendation in Doppler [7]. Prior cloud provisioning and right-sizing work studies data-driven VM selection, temporal bin packing, and predictive capacity planning under cloud catalogs [2, 4, 17, 23]. These studies motivate comparison dimensions; they neither establish the behavior of this article’s paper-side replay baselines nor serve as direct experimental baselines here.

2.3 Positioning of the Empirical Contribution

The three evidence categories have distinct roles. Capacity-reservation theory, especially Chen et al. [9], supplies analytical policy structure and price-aware commitment logic. Published resource-management systems supply the autoscaling, workload-prediction, and SKU-recommendation mechanisms reported in their respective studies [7, 10, 18]. This article contributes neither new reservation theory nor a replication of commercial tools. Its contribution is an artifact-backed empirical comparison among one implemented percentile-based sizing pipeline, a price-basis and candidate-catalog audit, restored public traces with checksums and intervals, and simplified theory-derived and recency-window reservation simulations. Relative to Chen et al. [9], who provide the analytical reservation structure reused here, the added evidence is implementation- and trace-facing: feasibility boundaries, service decomposition, checksum-backed trace replay with bootstrap intervals, held-out policy replay, and explicit empirical-unit and evidence-boundary checks.

2.4 Forecasting for Capacity Planning

Forecasting remains useful for demand monitoring and anomaly context. Classical exponential smoothing, Prophet, SARIMAX-style models, and time-series evaluation metrics provide forecasting context [1, 13, 16, 20]. The generated artifacts calculate walk-forward MAPE, MASE, empirical interval coverage, and selected small-sample paired tests [11, 12, 14]; the compact article tables and interpretation below focus on MAPE and selected paired-test results. The current evidence does not make forecast superiority central to the cost claim: forecasts are useful monitoring context, while capacity choices in the reported studies are based on empirical utilization distributions and price comparisons.

3 Background and Theoretical Framing

3.1 Two Capacity Questions

The same utilization trace can answer two distinct questions:

- **Right-sizing:** What candidate capacity does the evaluated utilization mapping select?

- **Reservation sizing:** In theory, how much physical capacity K should be committed in advance; in Study III, what arm-specific empirical proxy quantity Q should be selected under the corresponding objective?

The implemented pipeline uses the conservative rule

$$r_i = q_{0.95}(u_i)V_i h, \quad h = 1.3, \quad (1)$$

where u_i is observed utilization and V_i is current capacity. This capacity-sizing rule adds headroom above high observed usage.

For reservation sizing, the theoretical normalized cost of static reserved capacity K under physical demand D is

$$C(K) = \gamma K + \mathbb{E}(D - K)^+, \quad (2)$$

where D and K are expressed in physical-capacity units and $\gamma = w_T/w_0$ is the reserved/on-demand price ratio. The newsvendor solution is characterized by the $(1 - \gamma)$ quantile of the demand distribution; Study III uses the corresponding empirical training-sample quantile for each proxy arm. With $\gamma = 0.63$, this target is approximately 0.37, before any application-specific adjustment. A rule near P95 with 30% headroom therefore selects substantially more capacity than the price-aware static reservation quantity under this evaluated objective. This capacity difference does not establish an operational performance effect.

3.2 Savings Tradeoffs of P95 x 1.3 Under the Evaluated Model

The evaluated objectives treat high observations differently. A high right-sizing percentile assigns capacity using rare high usage, whereas a reservation contract saves money only when the discount on committed units exceeds the expected waste from units that are idle in the test period. As γ rises, the newsvendor fractile $1 - \gamma$ falls. The mapped-cost minimizer under this static objective reserves less because each committed unit becomes less attractive relative to on-demand. P95 x 1.3 does not use this price ratio and can select capacity above the mapped-cost-minimizing static quantity for the evaluated demand and prices.

This distinction also explains why higher apparent savings in a baseline grid are not automatically better policies. Mean, median, or lower-percentile sizing can reduce mapped costs more than P95 x 1.3, but the repository does not measure resulting SLO violations, latency changes, throughput, errors, or availability. The table therefore reports a mapped-cost/headroom tradeoff; it does not establish the operational safety of any row.

4 System and Methodology

4.1 Pipeline Scope

Figure 1 summarizes Smart Cloud Optimizer. Telemetry enters a SQLite-backed pipeline from a synthetic demo account or from public trace replays. The optimizer layer exposes forecasting, MILP right-sizing, reserved-pricing checks, and service-specific waste rules. The user interface and LLM-advisor components are not evaluated for savings in this article.

4.2 Forecasting and Monitoring

The forecasting module exposes a shared fit/predict interface for Naive, SeasonalNaive, ETS, Prophet, and SARIMAX-style models. The paper scripts report walk-forward MAPE and selected

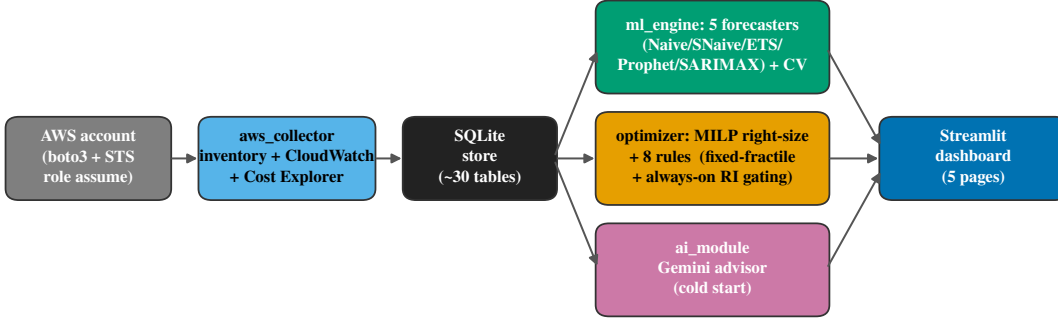


Figure 1: Smart Cloud Optimizer architecture. The evaluated research behavior is the right-sizing and reservation-policy logic, not dashboard presentation or LLM-advisor output.

paired tests where enough aligned errors are available. Forecasts are used as monitoring context. They are not the source of the reported right-sizing savings.

4.3 MILP Right-Sizing

For compute resources, the optimizer solves a binary assignment problem. Let $\mathcal{I}$ be resources and $\mathcal{C}$ candidate instance types with price p_c , vCPUs v_c , and memory m_c . The decision variable $x_{i,c} \in \{0, 1\}$ assigns resource i to candidate c :

$$\min_x \sum_{i \in \mathcal{I}} \sum_{c \in \mathcal{C}} p_c x_{i,c} \quad (3)$$

$$\text{s.t.} \quad \sum_c x_{i,c} = 1 \quad \forall i, \quad (4)$$

$$\sum_c v_c x_{i,c} \geq r_i^{cpu} \quad \forall i, \quad (5)$$

$$\sum_c m_c x_{i,c} \geq r_i^{mem} \quad \forall i, \quad (6)$$

$$\sum_{i,c} p_c x_{i,c} \leq B \quad \text{if a cap is supplied.} \quad (7)$$

EC2 uses CPU and memory when memory utilization is available. RDS uses CPU because the stored RDS memory metric is not directly comparable as a utilization percentage.

4.4 Rules and Reservation Checks

The rule engine adds transparent service-specific actions such as EC2/RDS reserved-pricing switches and waste-removal checks. These rules provide implementation context and a sanity check on the committed demo database. The paper’s quantitative claims come from the generated MILP evidence tables, the restored public-trace study, and reservation-policy simulations.

4.5 Price-Basis Normalization

Price comparability is central to the evaluated evidence. The committed synthetic database contains EC2 current-inventory costs that mostly match the 2026-06-30 AWS Price List snapshot for Linux,

shared-tenancy instances in `us-east-1`, but several EC2 candidate rows are incorrect or incomplete. Therefore, the analysis does not use the stale stored `recommendations` table as a headline result. Instead, EC2 current inventory and EC2 candidates are normalized to the same snapshot on-demand basis. Two current `m5.xlarge` rows that were originally recorded at one-year reserved-instance prices are evaluated on the shared on-demand basis for right-sizing comparability. This does not mean those instances would be billed as on-demand in a live account. Reserved-instance and other commitment effects are evaluated separately through the paper-side usage-hour RI replay and Study III.

RDS remains on the committed demo database price basis because the paper-local snapshot covers EC2 only. Consequently, the Study I aggregate result must always be read with the EC2/RDS split.

5 Experimental Design

5.1 Research Questions

The evaluation addresses four questions:

- RQ1: What does the implemented optimizer report on the repository’s synthetic AWS account after repairing the EC2 price basis?
- RQ2: What mapped-cost changes does the same MILP right-sizing rule produce on restored public VM traces under a consistent price catalog?
- RQ3: What do walk-forward MAPE and selected paired tests show about the forecasting models as monitoring context?
- RQ4: What is the held-out cost of counterfactually reusing a high-headroom right-sizing fractile for reservation sizing?

5.2 Datasets and Inputs

Table 1 summarizes the evaluated inputs. Study I uses the committed synthetic AWS account. Study II uses restored Bitbrains and Materna GWA traces [15, 19], priced against a snapshot-dated AWS catalog. The Azure Public Dataset V2 arm is retained only as a generated boundary condition because the raw `vmtable.csv.gz` input was not restored in this workspace.

5.3 Metrics and Baselines

Right-sizing results are reported as estimated monthly cost changes relative to the relevant baseline. Study I compares the current synthetic inventory against generated sizing policies. Study II compares each real-trace mapping against the cheapest AWS type that covers provisioned cores and memory. Forecasting reports walk-forward MAPE and selected paired-test results where aligned errors are available. Study III reports held-out policy cost as a percentage of an on-demand-only baseline.

Study I confidence intervals use 5,000 paired resource-level bootstrap resamples over the 10 compute resources with seed 20260709. Study II uses 2,000 paired VM-level bootstrap resamples with the same seed. These intervals describe heterogeneity across the evaluated resources under the fixed price mapping; they are not population-level confidence statements. Policy replay is

Table 1: Evaluation inputs and artifact status. GWA checksums and provenance are recorded in `paper/trace_provenance.md`; the Azure raw-input checksum is unavailable until `vmtable.csv.gz` is restored.

Input	Scale and period	Used for	Artifact files
Synthetic AWS account	365 days (2024-07-01–2025-06-30), 42 resources	optimizer savings, forecasting, policy simulation	<code>paper/results_baselines.json</code> ; <code>paper/results_ablations.json</code> ; <code>paper/numbers_evidence.tex</code>
Bitbrains fastStorage	1250 VMs, 2013 trace	real-trace MILP right-sizing	<code>paper/results_external.json</code> ; <code>paper/trace_provenance.md</code>
Bitbrains Rnd	500 VMs, 2142 hourly aggregate points	right-sizing, forecasting, policy simulation	<code>paper/trace_rnd_hourly.csv</code> ; <code>paper/results_policy_ext.json</code>
Materna-Trace-3	547 VMs, 2016 trace	second-provider right-sizing validation	<code>paper/results_external.json</code> ; <code>paper/table_external_baselines.tex</code>
Azure Public Dataset V2	237,272 filtered VMs from 2,695,548 rows, 2019 trace	recorded boundary case	<code>paper/results_azure.json</code> ; raw checksum pending

deterministic for a fixed chronological split and is supplemented by 50/60/70% split sensitivity rather than confidence intervals.

The baseline families are intentionally separated. Mean, median, percentile, and max right-sizing rows are mathematical baselines. Newsvendor, SAA, dual-capacity, and recency-window replay rows are paper-side reservation simulations. The synthetic usage-hour RI replay and Study III recency-window rows are not AWS Compute Optimizer, Cost Explorer, Azure Advisor, Savings Plans, or third-party recommender outputs.

6 Study I: Synthetic Account, Catalog Audit, and Service Decomposition

6.1 Catalog Audit and Feasibility Boundary

The synthetic account spans 365 days from 2024-07-01 to 2025-06-30. The pricing audit shows that the committed EC2 current-inventory costs mostly match the snapshot AWS catalog, but several committed EC2 candidate rows are wrong or incomplete. The original demo candidate set contains 13 EC2 price rows, but only four EC2 rows have the positive price and hardware specifications required by the MILP. Under $q_{0.95} \times 1.3$, the EC2 subproblem is therefore partial/infeasible in the original demo catalog while RDS remains feasible. This is a catalog-completeness boundary, not a universal claim about P95 sizing.

Table 2 shows the normalized comparison. With a shared 28-type EC2 snapshot catalog, non-budget EC2 infeasibilities disappear. The remaining EC2 infeasibilities are confined to explicit budget-cap rows.

6.2 Baseline and Service-Level Results

Table 3 reports the generated synthetic baseline grid. The headline Study I row is the deployed $q_{0.95} \times 1.3$ policy on the shared EC2 snapshot basis. Current synthetic compute cost is \$1241.58/mo and the feasible P95 $\times 1.3$ row costs \$1005.65/mo, a compute-scope change of -19.0%.

Table 2: Catalog-comparison rerun for the synthetic-account EC2 feasibility boundary. The snapshot row uses a shared EC2 price basis: current EC2 inventory and EC2 candidates are repriced with the documented AWS Price List snapshot in `paper/aws_catalog.py`; RDS candidates remain from the committed demo database. The 19.0% snapshot-row saving is the service-mixed aggregate decomposed in Table 4. Non-cap EC2 infeasible rows exclude explicit budget-cap constraints. Partial rows are not comparable savings estimates.

Catalog/basis	EC2 raw	EC2 valid	P95 $\times$ 1.3 status	Cost (\$/mo)	Saving (%)	Non-cap infeas. rows	Cap infeas. rows	Boundary
Original demo DB	13	4	Partial (EC2 Infeasible)	–	–	9/18	4/4	catalog coverage
AWS snapshot shared EC2 basis	28	28	Optimal	1,006	19.0	0/18	4/4	service budget cap

EC2 infeasibility disappears for non-budget baseline and ablation rows with the shared-basis snapshot EC2 catalog; remaining EC2 infeasibility is confined to explicit per-service budget-cap rows.

Table 3: Generated synthetic-account baseline comparison. Synthetic compute rows use a shared price basis: EC2 current inventory and EC2 candidates are repriced with the snapshot-dated AWS catalog in `paper/aws_catalog.py`; RDS remains on the committed demo DB basis. Partial or failed rows are not comparable savings estimates and are shown as “–”.

Method	Resources	Cost (\$/mo)	Saving (%)	95% CI (%)	Down	Up	Status
Current inventory	10	1,242	0.0	[0.0, 0.0]	0	0	Baseline
Mean utilization	10	347	72.0	[48.0, 85.5]	9	0	Optimal
Median utilization	10	332	73.3	[50.5, 86.4]	9	0	Optimal
P90 sizing	10	660	46.8	[17.5, 74.9]	9	0	Optimal
P95 sizing	10	660	46.8	[17.5, 74.9]	9	0	Optimal
P95 $\times$ 1.3 heuristic	10	1,006	19.0	[-48.0, 72.5]	8	2	Optimal
P99 sizing	10	660	46.8	[17.5, 74.9]	9	0	Optimal
Max utilization	10	736	40.7	[10.3, 69.2]	7	0	Optimal

Table 4 is the required interpretation of that aggregate. EC2 increases from \$843.73/mo to \$970.61/mo, a \$126.87/mo (15.0%) increase. RDS decreases from \$397.85/mo to \$35.04/mo, a \$362.81/mo (91.2%) decrease. Because RDS remains demo-priced, the aggregate -19.0% result is method-illustrative. It is not an unqualified optimizer win and should not be read as production-dollar evidence.

The P95 $\times$ 1.3 aggregate bootstrap interval, $[-48.0\%, 72.5\%]$, spans both cost increase and cost reduction. Its width is expected for only 10 heterogeneous resources and a service mix in which the EC2 and RDS effects have opposite signs. The point estimate is therefore a transparent case-study summary, not evidence that the same aggregate saving would generalize to a production fleet.

6.3 Ablations, Runtime, and Rule Sanity Checks

Table 5 reports the full synthetic ablation grid. The percentile and headroom rows show a mapped-cost/headroom tradeoff: lower requirements can produce higher apparent mapped savings, while P95 $\times$ 1.3 selects more candidate capacity. These rows do not measure operational performance effects. The table also keeps budget infeasibilities visible rather than converting fallback diagnostics

Table 4: Study I service-level decomposition for the generated $q_{0.95} \times 1.3$ row. EC2 current inventory and candidates use the shared snapshot on-demand basis; RDS remains on the committed demo DB basis. Positive deltas indicate cost increases.

Scope	Current (\$/mo)	Optimized (\$/mo)	Delta (\$/mo)	Delta (%)
EC2	844	971	+127	+15.0
RDS	398	35	-363	-91.2
Aggregate	1,242	1,006	-236	-19.0

Table 5: Generated ablation results on the synthetic account with EC2 repriced to the snapshot AWS catalog. Percentile, headroom, memory, and budget rows report assignment cost only for feasible MILP solves; aggregate 19.0% rows inherit the EC2/RDS split and RDS demo-price caveat in Table 4. Partial or failed rows expose feasibility limits and are not comparable savings estimates. Rule-set rows are a recomputed implementation sanity check on the committed demo DB, not the source of headline paper savings.

Dimension	Level	Cost (\$/mo)	Saving (%)	CPU exceed (%)	Recs/vars	Status
percentile q	0.75	478	61.5	1.2	232	Optimal
percentile q	0.90	1,006	19.0	0.0	232	Optimal
percentile q	0.95	1,006	19.0	0.0	232	Optimal
percentile q	0.99	1,141	8.1	0.0	232	Optimal
headroom h	1.0	660	46.8	0.0	232	Optimal
headroom h	1.1	660	46.8	0.0	232	Optimal
headroom h	1.3	1,006	19.0	0.0	232	Optimal
headroom h	1.5	1,141	8.1	0.0	232	Optimal
memory constraint	off	585	52.9	0.0	232	Optimal
memory constraint	observed	1,006	19.0	0.0	232	Optimal
budget cap	none	1,006	19.0	0.0	232	Optimal
budget cap	100% current	—	—	—	232	Partial (EC2 Infeasible)
budget cap	90% current	—	—	—	232	Partial (EC2 Infeasible)
budget cap	75% current	—	—	—	232	Partial (EC2 Infeasible)
budget cap	1% current	—	—	—	232	Failed (EC2 Infeasible, RDS Infeasible)
rule set	MILP right-sizing only	—	—	—	2	\$362.81/mo
rule set	reserved pricing only	—	—	—	2	\$125.85/mo
rule set	waste rules only	—	—	—	11	\$19.17/mo
rule set	all rules, deduplicated	—	—	—	15	\$507.83/mo

into comparable savings.

Table 6 records runtime and clone-stress diagnostics for the generated evidence script. These are not a hardware-normalized scalability study, but they document solver status, solve counts, and EC2 MILP stress sizes used by the current artifact.

The recomputed implementation sanity check emits 15 deduplicated recommendations worth \$507.83/mo under the committed demo database path. This is retained as system-context evidence only. The stale stored 19-row recommendation table is not used as a headline result.

Table 6: Runtime and clone-stress diagnostics for the generated evidence script. GWA Bit-brains/Materna raw-trace parsing runtimes are recorded in `paper/trace_provenance.md`; Azure raw parsing runtime is pending restoration of `vmtable.csv.gz`. These diagnostics are not a hardware-normalized scalability study.

Experiment	n	Vars	Solves	Solve s	Peak MB	Status
baseline sweep	10	1624	14	0.117	114.1	computed
ablation sweep	10	3480	30	0.248	118.6	computed
rule-set ablation	10	0	0	0.557	118.6	computed
EC2 MILP stress n=100	100	2800	1	0.071	120.4	Optimal
EC2 MILP stress n=500	500	14000	1	0.396	141.5	Optimal
EC2 MILP stress n=1000	1000	28000	1	1.285	171.4	Optimal
evidence script total	10	49904	47	7.860	171.4	computed

Table 7: Paper-side EC2 usage-hour RI replay baseline. The replay uses trailing observed metric coverage and the same snapshot EC2 on-demand/reserved prices; it is not AWS Compute Optimizer, Cost Explorer, Azure Advisor, or third-party recommender output. The 7-, 30-, and 60-day windows collapse to one row because they select the same six fully observed synthetic EC2 instances.

Recency windows (days)	Recs	Current (\$/mo)	Saving (\$/mo)	Saving (%)	Status
7/30/60	6	563	209	37.1	paper-side replay

Table 7 reports the paper-side synthetic EC2 usage-hour RI replay baseline. The 7-, 30-, and 60-day windows collapse to the same row because the same six eligible on-demand EC2 instances have complete metric coverage in every window and the snapshot reserved/on-demand prices are static. This table is not third-party recommender output.

6.4 Synthetic Forecasting Context

Forecasting is useful for monitoring, but not decisive for the cost claims. Table 8 shows walk-forward MAPE on the synthetic account. ETS has the best 30-day mean (11.2%), but the ETS versus seasonal-naive difference is not statistically significant at the reported sample size ($p = 0.198$). The paper therefore treats ETS as a selected monitoring model, not proof of forecast-driven optimization superiority.

Table 8: Study I synthetic-account walk-forward MAPE (% , mean $\pm$ sd). Values are transcribed unchanged from the table generated by `paper/make_figures.py`.

Model	$h=7$	$h=14$	$h=30$	$h=60$	$h=90$
ETS (selected)	11.28 ± 2.54	11.24 ± 2.45	11.20 ± 0.57	11.39 ± 0.74	12.15 ± 2.39
SeasonalNaive	10.18 ± 2.84	10.84 ± 2.75	12.28 ± 2.82	12.66 ± 2.21	12.91 ± 2.02
Prophet	14.41 ± 6.22	20.47 ± 7.40	20.93 ± 7.53	19.70 ± 7.89	18.08 ± 8.42
Naive	25.16 ± 4.47	27.90 ± 3.52	27.26 ± 4.36	27.87 ± 4.09	27.38 ± 3.70

7 Study II: Restored GWA Real-Trace Validation

7.1 Restoration and Cost Model

Study II was restored and rerun from raw GWA inputs. The artifact records archive-content SHA256 hashes for the Bitbrains fastStorage zip, Bitbrains Rnd zip, and Materna zip, plus filename-and-size manifest SHA256 hashes for the extracted CSV trees. The rerun uses a snapshot-dated AWS EC2 catalog from 2026-06-30 with 28 candidate types. Each VM baseline maps provisioned cores and memory to the cheapest covering AWS type. The optimized assignment uses the same deployed $q_{0.95} \times 1.3$ requirement.

Table 9 summarizes the restored inputs. It reports 16-hex prefixes for readability; the complete archive-content and filename-and-size manifest SHA256 values are preserved in `paper/trace_provenance.md` and `paper/results_external.json`.

The recorded rerun wall times were 176.06 seconds for fetch/extraction, 169.12 seconds for external validation, 127.34 seconds for the checksum, raw-quantile-scan, and baseline-grid helper stages, and 4.51 seconds for the real-trace policy simulation. These timings document the artifact run; they are not a hardware-normalized scalability benchmark, and a full process-memory and dependency-version inventory remains pending.

The absolute dollar values are mapped costs, not observed AWS bills. The GWA traces are older managed-hosting traces priced against a 2026 AWS catalog. The percentages provide within-model comparisons under this shared mapping.

7.2 Right-Sizing Results with Confidence Intervals

Table 10 reports the restored GWA right-sizing results under the evaluated mapping. On Bitbrains fastStorage and Rnd, the mapped-cost reductions are 33.6% (95% CI 24.5–41.8%) and 33.1% (19.3–46.3%), respectively. On Materna, the mapped reduction is 74.2% (71.4–76.8%). Across all restored GWA VMs, the aggregate mapped reduction is 41.8% (35.9–47.5%). Applying one-year reserved pricing to the optimized always-on fleet raises the aggregate mapped reduction to 63.4% (59.7–67.0%).

7.3 Full Baseline Grid and Mapped-Cost/Headroom Interpretation

Table 11 gives the full restored-GWA baseline grid with costs and bootstrap intervals. Mean, median, and less-conservative percentile sizing can produce higher apparent mapped savings than $q_{0.95} \times 1.3$. These rows are not automatically better policies because the repository does not evaluate SLO, latency, error, throughput, availability, or other operational outcomes after right-sizing. The table exposes a mapped-cost/headroom tradeoff: P95 $\times 1.3$ selects larger candidate capacities and

Table 9: Restored GWA provenance summary. Archive hashes cover archive contents; manifest hashes cover only relative filenames/paths and file sizes. The 1,500 Rnd CSV files represent the same 500 VMs across three monthly directories. The Materna archive contains 1,593 CSV files; the evaluation uses 547 VMs from Trace 3.

Archive	Bytes	Extracted CSVs	Archive SHA256 prefix	Filename/size manifest SHA256 prefix
fastStorage	134,047,207	1,250	11313f528a0cbcbe	63e26df5c3af611b
Rnd	163,099,172	1,500	d3d9ddebb689c0b5	5e288d1ae5c7cb8e
Materna	145,292,565	1,593	1380879f0de17cb5	07ff13555d88df59

Table 10: Study II restored-GWA right-sizing at AWS on-demand rates from snapshot 2026-06-30. The +RI column applies one-year reserved pricing.

Dataset	VMs	Baseline (\$/mo)	Optimized (\$/mo)	Saving / +RI (%)
fastStorage	1250	128,588	85,414	33.6 / 58.3
Rnd	500	44,294	29,614	33.1 / 58.0
Materna	547	44,184	11,398	74.2 / 83.8

yields lower mapped savings than several rows. Added headroom is not evidence of operational performance safety.

7.4 Azure Boundary Condition

The Azure Public Dataset V2 result is retained as a boundary condition, not as a rerun claim. The raw `vmtable.csv.gz` file was not present in this workspace, so the Azure arm was not restored, checksummed, rerun, or extended with confidence intervals. The committed generated artifact contains 237,272 filtered VMs from 2,695,548 rows. Under the recorded Azure preprocessing and metric-semantics assumptions, the mapped right-sizing result was -3.8%, while the recorded result with reserved pricing was 34.8%. Because the raw Azure input was not restored and those assumptions were not independently ablated, this result identifies a boundary case but does not establish which assumption caused the reversal.

7.5 Real-Trace Forecasting Context

Table 12 reports the Rnd aggregate forecast errors, and Figure 2 shows the series used for forecasting and policy replay. Forecasting the real aggregate is difficult. At 24 hours, the best hourly model is Prophet with 60.8% MAPE, and tested pairwise differences are not statistically significant (for example, ETS vs. seasonal-naive $p = 0.232$). The evidence supports treating forecasts as monitoring context rather than the source of right-sizing savings.

Table 11: Full Study II restored-GWA baseline grid. Savings are relative to the lift-and-shift mapping and include bootstrap 95% intervals.

Dataset	Policy	Cost (\$/mo)	Saving [95% CI]
fastStorage	Lift-and-shift	128,588	0.0 [0.0, 0.0]
fastStorage	Mean	23,920	81.4 [79.0, 83.6]
fastStorage	Median	16,973	86.8 [85.0, 88.3]
fastStorage	P90	68,673	46.6 [38.8, 53.9]
fastStorage	P95	76,845	40.2 [32.0, 48.0]
fastStorage	P99	84,908	34.0 [25.6, 41.8]
fastStorage	Max	119,196	7.3 [-1.1, 15.4]
fastStorage	P95 x 1.3	85,414	33.6 [24.5, 41.8]
Rnd	Lift-and-shift	44,294	0.0 [0.0, 0.0]
Rnd	Mean	7,728	82.6 [79.4, 85.5]
Rnd	Median	4,273	90.4 [88.7, 91.9]
Rnd	P90	25,697	42.0 [28.6, 54.3]
Rnd	P95	27,720	37.4 [23.5, 50.3]
Rnd	P99	30,088	32.1 [17.9, 45.0]
Rnd	Max	45,947	-3.7 [-18.8, 10.6]
Rnd	P95 x 1.3	29,614	33.1 [19.3, 46.3]
Materna	Lift-and-shift	44,184	0.0 [0.0, 0.0]
Materna	Mean	6,141	86.1 [84.7, 87.3]
Materna	Median	5,974	86.5 [85.1, 87.7]
Materna	P90	8,223	81.4 [79.4, 83.2]
Materna	P95	9,370	78.8 [76.5, 80.9]
Materna	P99	11,786	73.3 [70.5, 75.9]
Materna	Max	25,300	42.7 [38.3, 47.3]
Materna	P95 x 1.3	11,398	74.2 [71.4, 76.8]

Table 12: Study II real Rnd aggregate forecasting MAPE (% , mean $\pm$ sd).

Model	$h=6$ h	$h=12$ h	$h=24$ h	$h=48$ h	$h=7$ d	$h=14$ d
ETS	48.2 $\pm$ 46.2	71.9 $\pm$ 55.4	77.0 $\pm$ 57.2	91.2 $\pm$ 47.8	52.1 $\pm$ 29.9	64.4 $\pm$ 40.9
SeasonalNaive	65.9 $\pm$ 33.6	73.7 $\pm$ 46.3	65.0 $\pm$ 40.3	65.8 $\pm$ 33.8	54.2 $\pm$ 22.6	59.7 $\pm$ 32.9
Prophet	49.0 $\pm$ 26.2	60.5 $\pm$ 30.0	60.8 $\pm$ 21.6	88.1 $\pm$ 42.3	59.8 $\pm$ 36.9	66.7 $\pm$ 30.0
Naive	57.4 $\pm$ 71.2	87.9 $\pm$ 86.1	82.3 $\pm$ 66.6	94.2 $\pm$ 55.7	53.4 $\pm$ 14.2	66.5 $\pm$ 44.5

8 Study III: Capacity-Reservation Policy Simulation

8.1 Protocol

Study III isolates the reservation-sizing question. Each simulation splits an empirical proxy series X_t chronologically into train and test segments, selects a commitment proxy Q from the training data, and charges that policy on the held-out suffix. The synthetic run uses 219 training days and 146 test days. The restored real-trace run uses 1285 training hours and 857 test hours from the Bitbrains Rnd aggregate.

The deployed $q_{0.95} \times 1.3$ right-sizing rule is reused counterfactually as a naive reservation-sizing rule. This is neither deployed reservation behavior nor a model of AWS, Azure, or commercial recommendation systems. The counterfactual quantifies mapped cost when the algebra of a high-headroom sizing rule is applied to a commitment-proxy decision.

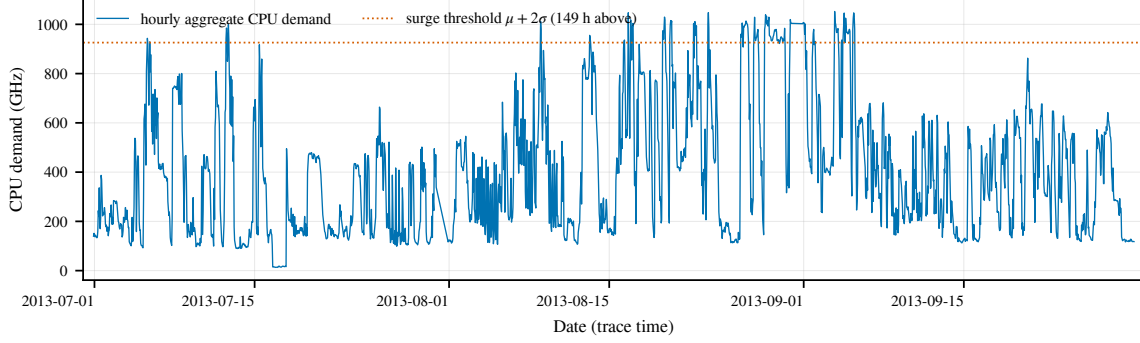


Figure 2: Bitbrains Rnd aggregate demand series used for real-trace forecasting context and Study III policy replay.

8.2 Empirical Mapping and Units

The theoretical formulation in Section 3 expresses D_t and K in physical-capacity units. The empirical simulations preserve the objective’s algebra but use arm-specific quantities X_t and Q to avoid implying that both inputs are physical capacity. In the synthetic arm, X_t is the daily account-cost series read from `total_cost`; both X_t and Q are measured in USD/day. This arm evaluates relative temporal policy behavior on a cost-proxy time series. It does not simulate provider billing or physical-capacity allocation directly.

In the restored real-trace arm, X_t is the hourly aggregate CPU demand constructed by summing per-VM hourly-mean CPU use, and X_t and Q are measured in MHz. The objective then has on-demand-rate-normalized, MHz-equivalent mapped-cost units per hourly period; it is not an observed cloud bill or a resource-level allocation simulation. Reported percentages divide each arm’s mapped cost by its own on-demand-only baseline and are dimensionless. Absolute Q values and mapped-cost units are not compared across arms.

For an empirical training sample S_X , the static proxy objective is the sample analogue of the theoretical model:

$$\hat{C}_\gamma(Q; S_X) = \gamma Q + \frac{1}{|S_X|} \sum_{x \in S_X} (x - Q)^+. \quad (8)$$

Every non-clairvoyant policy is sized only on the chronological training prefix and is then charged on the untouched test suffix. The policy set is:

- **OD:** on-demand only, $Q = 0$.
- **HEUR:** $Q = 1.3q_{0.95}(S_X)$, the deployed right-sizing rule read counterfactually as proxy commitment sizing.
- **NV:** $Q = q_{1-\gamma}(S_X)$, the empirical newsvendor solution.
- **SAA:** a PuLP/CBC linear program that minimizes $\hat{C}_\gamma$ over the training scenarios. It represents the same static objective as NV and is used as an implementation cross-check, not as an independent adversarial baseline.
- **LOOK7/30/60:** the NV/SAA objective fitted to only the trailing 7, 30, or 60 days of training data. These are 7/30/60 daily observations in the synthetic arm and 168/ 720/1285 hourly observations in the real-trace arm. If the requested window exceeds the training prefix, the

implementation uses the complete prefix. These rows are paper-side recency-window baselines, not evidence about recommendation systems.

- **DUAL/DUALC:** a two-level SAA with always-reserved base Q_0 , supplementary proxy commitment $Q_1 - Q_0$, and a training threshold of the mean plus two standard deviations. DUAL observes the contemporaneous threshold state and is an information-rich comparator; DUALC activates the same supplementary level from the previous period’s threshold signal.
- **LB:** The perfect-information lower-bound cost is $C_{LB} = \gamma \mathbb{E}[X_{\text{test}}]$, obtained by allowing a period-specific proxy commitment $Q_t = X_t$. Its units are USD/day proxy units in the synthetic arm and on-demand-rate-normalized MHz-equivalent units per hourly period in the real-trace arm. The bound is unattainable under advance commitment because it uses each future X_t to choose Q_t .

8.3 Synthetic and Real-Trace Proxy Results

At the evaluated catalog-average ratio $\gamma = 0.63$, the high-percentile counterfactual realizes 101.8% of on-demand cost on the synthetic USD/day proxy and 153.5% on the real-trace MHz proxy. The newsvendor fractile realizes 73.6% and 86.2%. This is the mapped-cost gap from counterfactually reusing a high-headroom sizing rule as proxy commitment sizing under the evaluated model. Table 13 reports the complete policy comparison at this ratio.

To avoid comparing the price-aware policies only against the deliberately conservative HEUR counterfactual, we include LOOK7, LOOK30, and LOOK60 as paper-side recency-window replay baselines. They are not outputs of, or evidence about, third-party recommendation systems. On the real trace at $\gamma = 0.63$, the 7-day replay realizes 91.2%, the 30-day replay realizes 84.9%, and the 60-day replay realizes 86.2% of on-demand cost. The LOOK60 row equals the newsvendor row in the real-trace grid because the requested 60-day window exceeds the available training split; the implementation falls back to the full training set, making LOOK60 use the same empirical distribution as the newsvendor policy.

The dual-capacity rows are close to NV on the synthetic daily series and have lower mapped cost than NV on the real trace under the evaluated simplified model and price ratios. This observational comparison does not establish why the difference occurs. The simulation remains an aggregate model, not a full dynamic (s, S) contract implementation.

The split-sensitivity replay shows how the mapped-cost ordering varies across alternative chronological splits. Across 50/60/70% training fractions, newsvendor saves 26.1–26.4% on the synthetic series and 12.5–16.0% on the real series. The corresponding HEUR-minus-NV gaps are 22.6–28.2 and 45.8–116.4 on-demand-cost percentage points. These are deterministic alternative splits, not confidence intervals.

Figure 3 shows both proxy-arm policy curves across the full price-ratio sweep. Tables 14 and 15 give the exact grid at four representative ratios.

Table 13: Held-out policy cost at the evaluated catalog-average $\gamma = 0.63$, as a percentage of on-demand-only cost. LOOK rows are recency-window replay baselines; DUAL observes contemporaneous state; LB is an unattainable perfect-information cost lower bound.

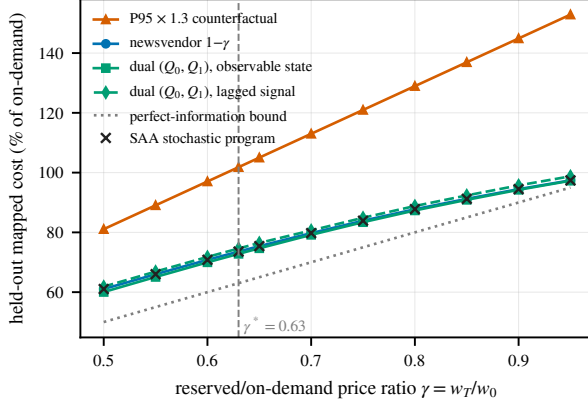
Policy	Synthetic cost-proxy arm	Restored Rnd proxy arm
On-demand only	100.0	100.0
P95 x 1.3 counterfactual	101.8	153.5
Newsvendor	73.6	86.2
SAA	73.6	86.2
LOOK7	74.1	91.2
LOOK30	73.3	84.9
LOOK60	73.4	86.2
Dual, observed state	72.8	77.7
Dual, lagged signal	74.6	78.1
Perfect-information bound	63.0	63.0

Table 14: Study III synthetic daily cost-proxy results as % of the on-demand-only baseline across the price-ratio grid. LOOK rows are paper-side recency-window baselines.

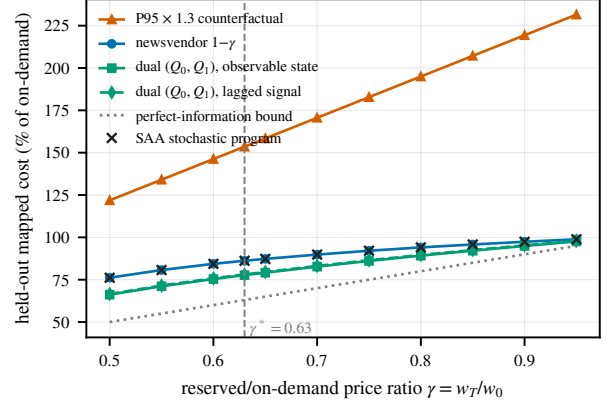
Policy	$\gamma=0.60$	$\gamma=0.70$	$\gamma=0.80$	$\gamma=0.90$
P95 $\times$ 1.3 counterfactual	97.0	113.0	128.9	144.9
newsvendor $1-\gamma$	70.8	79.7	87.7	94.5
SAA stochastic program	70.8	79.8	87.8	94.5
7-day recency-window replay	71.0	80.2	87.6	94.5
30-day recency-window replay	70.5	79.7	87.4	94.3
60-day recency-window replay	70.5	79.8	87.5	94.6
dual (Q_0, Q_1) , obs. state	70.0	79.2	87.3	94.2
dual (Q_0, Q_1) , lagged signal	71.8	80.6	88.8	95.7
perfect-information bound	60.0	70.0	80.0	90.0

Table 15: Study III real-trace hourly CPU-demand-proxy results as % of the on-demand-only baseline across the price-ratio grid. LOOK60 degenerates to NV because it uses the full training prefix.

Policy	$\gamma=0.60$	$\gamma=0.70$	$\gamma=0.80$	$\gamma=0.90$
P95 $\times$ 1.3 counterfactual	146.2	170.6	195.0	219.3
newsvendor $1-\gamma$	84.4	89.8	94.1	97.4
SAA stochastic program	84.4	89.8	94.1	97.4
7-day recency-window replay	89.3	95.6	102.4	103.6
30-day recency-window replay	82.7	89.2	93.8	97.3
60-day recency-window replay	84.4	89.8	94.1	97.4
dual (Q_0, Q_1) , obs. state	75.4	82.7	89.2	94.9
dual (Q_0, Q_1) , lagged signal	75.8	83.1	89.6	95.3
perfect-information bound	60.0	70.0	80.0	90.0



(a) Synthetic daily account-cost proxy (USD/day).



(b) Real-trace hourly aggregate CPU demand (MHz).

Figure 3: Study III held-out mapped cost as a percentage of each arm’s on-demand-only baseline across the price-ratio sweep. At $\gamma = 0.63$, the $P95 \times 1.3$ counterfactual exceeds the synthetic baseline and has higher mapped cost than the reported price-aware policies on both proxy arms. Panel (b)’s dual-level variants have lower mapped cost than NV on this trace under the simplified model. These curves are not provider-billing or physical-allocation simulations.

9 Discussion

9.1 What the Evidence Supports

The evaluated evidence supports three scoped observations. First, the implemented system operationalizes a percentile-and-headroom rule through MILP right-sizing and transparent service rules, subject to price-basis and candidate-catalog feasibility. Second, the documented artifact reruns report results from the synthetic database, restored public GWA traces, and a snapshot-dated price catalog. Third, a paper-side simulation that reuses inherited reservation logic reports different mapped costs for the high-headroom counterfactual and price-aware policies on two explicitly labeled proxy series.

The central empirical evidence is the combination of restored GWA right-sizing with confidence intervals and the held-out reservation-policy simulation, rather than the synthetic aggregate alone. Study I records price-basis and catalog-completeness issues in a controlled repository account. On the restored GWA traces, the same right-sizing rule produced the reported mapped-cost reductions relative to lift-and-shift under the evaluated catalog and mapping. Study III reports dimensionless within-arm cost ratios under the synthetic USD/day cost proxy, the real-trace MHz demand proxy, the static objective, and the evaluated price ratios. It does not convert either arm into provider-billing or deployed-allocation evidence.

9.2 What Should Not Be Oversold

The deployed system does not implement the full dynamic (s, S) policy of Chen et al. [9]. It implements static high-percentile right-sizing and always-on reserved-pricing checks. The newsvendor, SAA, dual-level, and recency-window results reuse inherited policy structure in paper-side simulations; they are not a theoretical extension. The LLM advisor is not evaluated for savings. The external baselines are mathematical or replay baselines, not direct runs of AWS, Azure, or third-party recommenders. The EC2 catalog comparison uses a documented 28-type snapshot, not an all-SKU AWS catalog.

The prior Azure aggregate remains a boundary case under recorded preprocessing and metric-semantics assumptions. Without the restored raw input or targeted ablations, it does not identify the cause of the mapped-cost reversal.

The higher-savings rows in Study II should also not be overread. They may be lower-cost under the mapping, but the repository does not measure latency, errors, throughput, availability, or SLO outcomes after right-sizing. The paper can therefore discuss a mapped-cost/candidate-capacity tradeoff, but it cannot claim that any row is operationally safe.

9.3 Implications for Practice

The practical implication is to separate the modeled decisions and their evidence. The implemented P95 x 1.3 rule adds capacity headroom and selects larger candidate sizes under the right-sizing mapping; the study does not establish whether that headroom improves latency, errors, throughput, availability, or SLO attainment. The paper-side reservation simulation separately accounts for the price ratio and proxy-series distribution. Under the evaluated series and price ratios, counterfactually reusing P95 x 1.3 produced higher mapped cost than the reported price-aware alternatives. That comparison remains a simulation result, not implemented provider behavior.

10 Threats to Validity

Synthetic data and demo pricing. Study I uses the repository’s synthetic AWS account. Its dollar values are method-illustrative and should not be presented as production savings. The aggregate P95 x 1.3 compute-scope change is dominated by demo-priced RDS while EC2 increases on the shared snapshot on-demand basis.

Price basis and catalog scope. EC2 right-sizing rows use a snapshot-dated 28-type catalog, not an all-SKU AWS catalog and not live AWS Compute Optimizer output. RDS remains on the committed demo database basis. Reserved-instance effects are separated from right-sizing.

Trace age and provider mapping. Bitbrains and Materna are older managed-hosting traces. They are mapped to a 2026 AWS catalog, so percentages are more defensible than absolute dollar values.

Azure raw-input gap. The Azure result remains a generated boundary condition because raw `vmtable.csv.gz` was absent. It needs restoration, checksum verification, rerun, confidence intervals, and runtime recording before it can carry the same evidentiary weight as the GWA study.

Azure preprocessing and metric semantics. Under the recorded assumptions, the Azure mapped result is a boundary case. Because the raw input was not restored and the assumptions were not independently ablated, the study cannot attribute the reversal to max-based CPU semantics, unavailable memory utilization, or any specific preprocessing choice.

Policy abstraction and empirical units. Study III uses a daily account-cost proxy in USD/day for the synthetic arm and hourly aggregate CPU demand in MHz for the real-trace arm. Its within-arm percentages are dimensionless, but its absolute proxy commitments are not comparable across arms. The simulation does not model provider billing, physical allocation, all real contract terms, partial upfront payments, cancellation, resale, family constraints, regional capacity, Savings Plans, or fragmentation.

Policy uncertainty. Study III is deterministic for each chronological split and reports split sensitivity, but it does not provide confidence intervals for policy-cost ratios. The dual-state results also depend on a mean-plus-two-standard-deviations surge threshold and should be read as model sensitivity rather than a production guarantee.

No direct third-party baseline. The usage-hour RI replay and recency-window rows are paper-side baselines only. The repository contains no direct AWS, Azure, or third-party recommender output on the same workload.

No SLO or performance-risk validation. The artifact reports mapped costs and limited exceedance proxies, but no live SLO, latency, error-rate, or business-performance outcomes. It therefore cannot claim safe performance after more aggressive right-sizing.

Statistical power. Forecasting tests use limited walk-forward folds. Non-significant differences remain non-claims.

11 Reproducibility and Artifact Availability

Most headline values are imported from generated artifacts, and the documented scripts provide rerun paths for the principal synthetic, restored-GWA, and policy results. Some compact presentation tables contain transcribed summaries or checksum prefixes. Synthetic evidence is generated by `paper/evidence_experiments.py` and `paper/catalog_comparison.py`; GWA evidence uses `paper/external_validation.py`, `paper/study2_reproducibility.py`, and the commands in `paper/trace_provenance.md`; policy results use `paper/policy_sim.py` and `paper/trace_rnd_hourly.csv`. The reference audit in `paper/reference_check.md` records that the 23 citation keys used by the source manuscript resolve. This article uses the same audited key set and introduces no new bibliography entries.

The principal rerun path is documented, but dependency pinning and clean-room verification remain pending. Azure is not reproducible in this workspace until `vmtable.csv.gz` is restored and rerun. Before public artifact release, the authors must add the repository URL and archive identifier, license, supported operating systems, Python and solver versions, dependency lock information, LaTeX engine version, execution hardware, and final generated-file hashes.

12 Conclusion

This study contributes an empirical and systems comparison, not new reservation theory. The implemented behavior is a $q_{0.95} \times 1.3$ MILP right-sizing pipeline whose reported outcomes depend on price comparability and candidate-catalog feasibility. On the synthetic account, the -19.0% service-mixed change combines an EC2 increase of 15.0% with a demo-priced RDS decrease of 91.2%. On restored Bitbrains and Materna traces, the same implemented rule produces 41.8% lower mapped cost than lift-and-shift under the evaluated catalog and mapping (95% CI 35.9–47.5%). The Study III results are separate paper-side simulations that reuse theory-derived logic on a synthetic USD/day cost proxy and a real-trace MHz demand proxy. They do not represent provider billing, commercial recommender behavior, or physical allocation. The Azure arm remains unrestored, and operational performance and SLO effects remain unresolved.

Declarations

Data availability. Synthetic data and generated outputs are in the repository. GWA restoration commands and checksums are recorded in `paper/trace_provenance.md`.

The Azure raw `vmtable.csv.gz` remains unrestored.

Code availability. Manuscript scripts are identified above by artifact-relative filenames in the paper artifact directory.

Author Contact Information

Corresponding author: Ibrahim Mohamed: ibrahimmohamedabdelsadek@gmail.com.

Author emails: Mariam Emad: mariamemadcs@gmail.com; Hazem Ibrahim: Ihazem28@gmail.com; Ahmed Sameh: ahmed.sameh.12543@gmail.com; Mahmoud Ahmed: mahmoudkamel19102003@gmail.com; John Ehab: ehabjohn22@gmail.com.

References

- [1] Ferran Agulló, Alberto Gutierrez-Torre, Jordi Torres, and Josep Ll. Berral. Enhancing the output of time series forecasting algorithms for cloud resource provisioning. *Future Generation Computer Systems*, 170:107833, 2025.
- [2] Moab Alves and Fabio Morais. IaaS capacity planning with multiple pricing models using predictive heuristics. In *2025 IEEE International Conference on Cloud Computing Technology and Science (CloudCom)*, pages 1–8, Shenzhen, China, 2025. IEEE.
- [3] Kenneth J. Arrow, Theodore Harris, and Jacob Marschak. Optimal inventory policy. *Econometrica*, 19(3):250–272, 1951.
- [4] Nürşen Aydın, İbrahim Mutur, and Ş. İlker Birbil. Multi-objective temporal bin packing problem: An application in cloud computing. *Computers & Operations Research*, 121:104959, 2020.
- [5] Gah-Yi Ban and Cynthia Rudin. The big data newsvendor: Practical insights from machine learning. *Operations Research*, 67(1):90–108, 2019.
- [6] Kerem Bülbül, Nilay Noyan, and Hazal Erol. Multi-stage stochastic programming models for provisioning cloud computing resources. *European Journal of Operational Research*, 288(3):886–901, 2021.
- [7] Joyce Cahoon, Wenjing Wang, Yiwen Zhu, Katherine Lin, Sean Liu, Raymond Truong, Neetu Singh, Chengcheng Wan, Alexandra Ciorrea, Sreraman Narasimhan, and Subru Krishnan. Doppler: Automated SKU recommendation in migrating SQL workloads to the cloud. *Proceedings of the VLDB Endowment*, 15(12):3509–3521, 2022.
- [8] Sivadon Chaisiri, Bu-Sung Lee, and Dusit Niyato. Optimization of resource provisioning cost in cloud computing. *IEEE Transactions on Services Computing*, 5(2):164–177, 2012.
- [9] Shi Chen, Junfei Lei, and Kamran Moinzadeh. Cost optimization in cloud computing: Capacity reservation for intermittent random demand surges. *Production and Operations Management*, 33(6):1265–1284, 2024. Also available as SSRN working paper 3784812.
- [10] Eli Cortez, Anand Bonde, Alexandre Muzio, Mark Russinovich, Marcus Fontoura, and Ricardo Bianchini. Resource Central: Understanding and predicting workloads for improved resource management in large cloud platforms. In *Proceedings of the 26th Symposium on Operating Systems Principles (SOSP)*, pages 153–167, 2017.
- [11] Francis X. Diebold and Roberto S. Mariano. Comparing predictive accuracy. *Journal of Business & Economic Statistics*, 13(3):253–263, 1995.

- [12] David Harvey, Stephen Leybourne, and Paul Newbold. Testing the equality of prediction mean squared errors. *International Journal of Forecasting*, 13(2):281–291, 1997.
- [13] Rob J. Hyndman and Yeasmin Khandakar. Automatic time series forecasting: The forecast package for R. *Journal of Statistical Software*, 27(3):1–22, 2008.
- [14] Rob J. Hyndman and Anne B. Koehler. Another look at measures of forecast accuracy. *International Journal of Forecasting*, 22(4):679–688, 2006.
- [15] Alexandru Iosup, Hui Li, Mathieu Jan, Shanny Anoop, Catalin Dumitrescu, Lex Wolters, and Dick H. J. Epema. The Grid workloads archive. *Future Generation Computer Systems*, 24(7):672–686, 2008.
- [16] Dionatră F. Kirchoff, Vinicius Meyer, Rodrigo N. Calheiros, and Cesar A. F. De Rose. Evaluating machine learning prediction techniques and their impact on proactive resource provisioning for cloud environments. *The Journal of Supercomputing*, 80(15):21920–21951, 2024.
- [17] Saira Musa and Douglas G. Down. Cost-effective cloud resource provisioning using linear regression. In *2024 IEEE International Conference on Cloud Computing Technology and Science (CloudCom)*, pages 176–183, Abu Dhabi, United Arab Emirates, 2024. IEEE.
- [18] Krzysztof Rządca, Paweł Findeisen, Jacek Swiderski, Przemysław Zych, Przemysław Broniek, Jarek Kusmierek, Paweł Nowak, Beata Strack, Piotr Witusowski, Steven Hand, and John Wilkes. Autopilot: Workload autoscaling at Google. In *Proceedings of the Fifteenth European Conference on Computer Systems (EuroSys ’20)*, pages 1–16. ACM, 2020.
- [19] Siqi Shen, Vincent van Beek, and Alexandru Iosup. Statistical characterization of business-critical workloads hosted in cloud datacenters. In *2015 15th IEEE/ACM Int. Symp. on Cluster, Cloud and Grid Computing (CCGrid)*, pages 465–474, 2015.
- [20] Sean J. Taylor and Benjamin Letham. Forecasting at scale. *The American Statistician*, 72(1):37–45, 2018.
- [21] Wei Wang, Ben Liang, and Baochun Li. Optimal online multi-instance acquisition in IaaS clouds. *IEEE Transactions on Parallel and Distributed Systems*, 26(12):3407–3419, 2015.
- [22] Wei Wang, Di Niu, Ben Liang, and Baochun Li. Dynamic cloud instance acquisition via IaaS cloud brokerage. *IEEE Transactions on Parallel and Distributed Systems*, 26(6):1580–1593, 2015.
- [23] Neeraja J. Yadwadkar, Bharath Hariharan, Joseph E. Gonzalez, Burton Smith, and Randy H. Katz. Selecting the best VM across multiple public clouds: A data-driven performance modeling approach. In *Proceedings of the 2017 Symposium on Cloud Computing (SoCC ’17)*, pages 452–465, New York, NY, USA, 2017. ACM.